

SQL Interview Preparation

Complete Guide — Concepts, Query Writing & Common Interview Problems

Candidate: Prashant Singh | Software Engineer (Full-Stack)

Scope: Core SQL, joins & keys, normalization, indexing, transactions, window functions, query-writing problems, optimization, and database design

Standard ANSI SQL — notes flag MySQL / PostgreSQL / SQL Server differences where they matter

- | | |
|------------------------------------|---|
| 1. SQL Fundamentals & Query Basics | 9. Indexes & Query Optimization |
| 2. Keys & Constraints | 10. Transactions, ACID & Locking |
| 3. Joins | 11. Views, Stored Procedures, Triggers, Functions |
| 4. Aggregation & Grouping | 12. NULL Handling, Strings & Dates |
| 5. Subqueries & CTEs | 13. SQL Injection & Security |
| 6. Window Functions | 14. SQL vs NoSQL |
| 7. Set Operations | 15. Classic Query-Writing Problems (with solutions) |
| 8. Normalization & Database Design | 16. Glossary of Terms |

How to use this guide: Every answer is written to be spoken aloud in an interview — concise but complete. Query-writing problems (Section 15) are the ones interviewers use most to test hands-on ability, so practice writing them from memory, not just reading them. A shared sample schema is used throughout for consistency.

Sample schema used in examples throughout this guide

```
Employees(emp_id, name, salary, dept_id, manager_id, hire_date)
Departments(dept_id, dept_name)
Orders(order_id, customer_id, order_date, amount)
Customers(customer_id, name, city)
```

1 SQL Fundamentals & Query Basics

Q What is SQL, and what are its main sub-languages?

A: SQL (Structured Query Language) is the standard language for defining, querying, and manipulating data in a relational database. It's split into: **DDL** (Data Definition Language — CREATE, ALTER, DROP, TRUNCATE — defines schema structure), **DML** (Data Manipulation Language — SELECT, INSERT, UPDATE, DELETE — reads/writes rows), **DCL** (Data Control Language — GRANT, REVOKE — manages permissions), and **TCL** (Transaction Control Language — COMMIT, ROLLBACK, SAVEPOINT — manages transactions).

Q What is the logical order in which a SQL query is actually executed?

A: Even though we write `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY`, the database evaluates it in this order:

1) `FROM / JOIN` → 2) `WHERE` → 3) `GROUP BY` → 4) `HAVING` → 5) `SELECT` → 6) `DISTINCT` → 7) `ORDER BY` → 8) `LIMIT/OFFSET`. This is why you can't reference a SELECT column alias inside WHERE (WHERE runs before SELECT), but you can in ORDER BY (which runs after).

Q What's the difference between WHERE and HAVING ?

A: `WHERE` filters individual rows *before* grouping/aggregation happens. `HAVING` filters *after* grouping — it operates on aggregated results, so it's the only place you can filter on something like `COUNT(*) > 5`.

```
SELECT dept_id, COUNT(*) AS emp_count
FROM Employees
WHERE hire_date >= '2024-01-01'
GROUP BY dept_id
HAVING COUNT(*) > 5;
```

Q What's the difference between DELETE, TRUNCATE, and DROP ?

A: `DELETE` is DML — removes rows one at a time (optionally with WHERE), is logged, can be rolled back, and fires triggers. `TRUNCATE` is DDL — deallocates all rows at once, resets identity/auto-increment counters, is minimally logged and much faster, but can't use WHERE and generally can't be rolled back in some databases. `DROP` removes the entire table object (structure, data, indexes, permissions) from the database.

Q What's the difference between UNION and JOIN ?

A: A JOIN combines columns from two tables side by side, based on a related key, producing wider rows. A UNION stacks the result sets of two queries vertically, combining rows from both into one result — the queries must have the same number of columns with compatible types.

Q What's the difference between a **clause**, **predicate**, and **expression** — and what are `LIMIT / OFFSET` vs `TOP` vs `FETCH` ?

A: Those are just syntax differences across databases for the same idea — restricting how many rows come back. MySQL/PostgreSQL use `LIMIT n OFFSET m` ; SQL Server uses `SELECT TOP n` or `OFFSET m ROWS FETCH NEXT n ROWS ONLY` ; Oracle traditionally used `ROWNUM` or now also supports `FETCH FIRST n ROWS ONLY` (the ANSI-standard form). It's worth naming the dialect you're most comfortable with in an interview.

2 Keys & Constraints

Q Explain Primary Key, Foreign Key, Unique Key, and Candidate Key.

A: A **candidate key** is any column (or set of columns) that could uniquely identify a row. The **primary key** is the candidate key chosen to be the table's official identifier — it's unique, can't be NULL, and a table has exactly one. A **unique key** also enforces uniqueness but can allow one NULL value and a table can have several. A **foreign key** is a column in one table that references the primary key of another table, enforcing referential integrity — you can't insert a foreign key value that doesn't exist in the parent table.

Q What is a composite key?

A: A primary key made up of two or more columns together, where no single column is unique on its own but the combination is. Example: in an `Enrollments(student_id, course_id, enrolled_date)` table, `(student_id, course_id)` together form the composite primary key, since a student can enroll in many courses and a course has many students, but each student-course pair appears once.

Q What database constraints do you know, and what does each enforce?

A:

Constraint	Enforces
<code>NOT NULL</code>	Column can't store a NULL value.
<code>UNIQUE</code>	All values in the column(s) must be distinct.
<code>PRIMARY KEY</code>	<code>NOT NULL</code> + <code>UNIQUE</code> , one per table, identifies each row.
<code>FOREIGN KEY</code>	Value must exist in the referenced table's key column.
<code>CHECK</code>	Value must satisfy a boolean condition, e.g. <code>CHECK (salary > 0)</code> .
<code>DEFAULT</code>	Supplies a value automatically when none is given on insert.

Q What happens if you try to delete a row that's referenced by a foreign key elsewhere?

A: By default the database rejects it with a referential-integrity violation, unless the foreign key was defined with a cascade rule: `ON DELETE CASCADE` (deletes the child rows too), `ON DELETE SET NULL` (nulls out the FK column in child rows), or `ON DELETE RESTRICT/NO ACTION` (blocks the delete, which is the default behavior).

3 Joins

Q Explain the different types of SQL joins.

A:

Join	Returns
INNER JOIN	Only rows with a match in both tables.
LEFT JOIN	All rows from the left table, matched columns from the right (NULL if no match).
RIGHT JOIN	All rows from the right table, matched columns from the left (NULL if no match).
FULL OUTER JOIN	All rows from both tables; unmatched sides filled with NULL.
CROSS JOIN	Cartesian product — every row of table A paired with every row of table B.
SELF JOIN	A table joined to itself, using aliases, to compare rows within the same table.

Q Write a query to list every employee with their department name, including employees with no department.

A:

```
SELECT e.name, d.dept_name
FROM Employees e
LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Q Write a self-join to list each employee alongside their manager's name.

A:

```
SELECT e.name AS employee, m.name AS manager
FROM Employees e
LEFT JOIN Employees m ON e.manager_id = m.emp_id;
```

Q How would you find rows that exist in the left table but have no match in the right table?

A: A LEFT JOIN followed by filtering where the right table's key is NULL — this is the standard "anti-join" pattern.

```
SELECT e.name
FROM Employees e
LEFT JOIN Departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

Q What's the difference between an implicit join (comma syntax with WHERE) and an explicit JOIN?

A: `SELECT * FROM A, B WHERE A.id = B.a_id` is the old implicit join syntax — it puts the join condition in WHERE, which is easy to forget and accidentally produces a cross join. Explicit `JOIN ... ON` syntax is preferred because it separates the join logic from filtering logic, is clearer to read, and makes outer joins (LEFT/RIGHT/FULL) actually expressible — the comma syntax can only do inner joins.

4 Aggregation & Grouping

Q What are the standard SQL aggregate functions, and how do they handle NULLs?

A: `COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()`. All of them (except `COUNT(*)`) ignore NULL values entirely — they don't treat NULL as zero. `COUNT(*)` counts all rows regardless of NULLs; `COUNT(column)` only counts rows where that column is non-NULL. This matters for `AVG()` especially, since NULLs are excluded from both the sum and the divisor.

Q Write a query to find the department with the highest average salary.

A:

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM Employees
GROUP BY dept_id
ORDER BY avg_salary DESC
LIMIT 1;
```

Q Why can't you reference a non-aggregated, non-grouped column in SELECT alongside GROUP BY?

A: Because GROUP BY collapses many rows into one per group, so for any column not in the GROUP BY list or wrapped in an aggregate, the database wouldn't know which of the group's many values to return — it's ambiguous. Strict SQL databases (like PostgreSQL) reject this at parse time; MySQL historically allowed it and silently picked an arbitrary row's value, which is a common source of subtle bugs.

5 Subqueries & CTEs

Q What's the difference between a subquery and a JOIN, and when would you prefer one over the other?

A: A subquery is a query nested inside another query, often used to filter (`WHERE id IN (SELECT ...)`) or compute a scalar value. A JOIN combines rows from two tables directly into the result set. I prefer JOINS when I need columns from both tables in the output, since the optimizer usually handles them more efficiently and they avoid re-executing correlated subqueries per row. I use subqueries when I only need to test membership/existence or compute a single derived value, and don't need columns from the other table.

Q What's a correlated subquery, and why can it be slow?

A: A correlated subquery references a column from the outer query, so it can't be run once and cached — conceptually, it re-executes for every row the outer query processes. Example: finding employees earning more than their department's average.

```
SELECT e.name
FROM Employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM Employees e2
    WHERE e2.dept_id = e.dept_id
);
```

Modern query optimizers often rewrite these into joins internally, but on large tables without that optimization, they can be much slower than an equivalent JOIN with a window function.

Q What is a CTE (Common Table Expression), and why use one instead of a subquery?

A: A CTE, defined with `WITH name AS (...)`, is a named, temporary result set scoped to a single query. It's functionally similar to a subquery, but it's more readable (especially when reused multiple times in the same query), can be chained/referenced by later CTEs, and is required for recursive queries — a plain subquery can't reference itself.

```
WITH dept_avg AS (  
  SELECT dept_id, AVG(salary) AS avg_sal  
  FROM Employees  
  GROUP BY dept_id  
)  
SELECT e.name, e.salary, d.avg_sal  
FROM Employees e  
JOIN dept_avg d ON e.dept_id = d.dept_id  
WHERE e.salary > d.avg_sal;
```

Q Write a recursive CTE to find all employees under a given manager (an org chart hierarchy).

A:

```
WITH RECURSIVE org_chart AS (  
  SELECT emp_id, name, manager_id, 1 AS level  
  FROM Employees  
  WHERE manager_id IS NULL      -- anchor: the top-level manager  
  
  UNION ALL  
  
  SELECT e.emp_id, e.name, e.manager_id, oc.level + 1  
  FROM Employees e  
  JOIN org_chart oc ON e.manager_id = oc.emp_id  -- recursive step  
)  
SELECT * FROM org_chart ORDER BY level;
```

The anchor member finds the base case, then the recursive member repeatedly joins back to the growing result until no new rows are produced.

6 Window Functions

Q What is a window function, and how is it different from GROUP BY?

A: A window function performs a calculation across a set of related rows (a "window", defined by `OVER (PARTITION BY ... ORDER BY ...)`) but, unlike GROUP BY, doesn't collapse those rows into one — each original row is kept, with the computed value attached. That's why window functions are used when you need both the detail rows and an aggregate/ranking value side by side.

Q Explain `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()` — what's the difference?

A: All three number rows within a partition based on an ORDER BY. `ROW_NUMBER()` always assigns unique, sequential numbers (1,2,3,4...) even for tied values. `RANK()` gives tied rows the same rank but skips subsequent numbers (1,2,2,4...). `DENSE_RANK()` gives tied rows the same rank without skipping (1,2,2,3...).

Q Write a query to find the 2nd highest salary in each department.

A:

```
SELECT emp_id, name, dept_id, salary
FROM (
  SELECT emp_id, name, dept_id, salary,
         DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
  FROM Employees
) ranked
WHERE rnk = 2;
```

DENSE_RANK is the right choice here (over ROW_NUMBER) so that two employees genuinely tied for highest salary don't cause the "2nd highest" to just be a duplicate of the top value.

Q What do LEAD() and LAG() do? Give an example use case.

A: LAG(col, n) looks back n rows within the window; LEAD(col, n) looks forward n rows — both without a self-join. A classic use: comparing each order's amount to the customer's previous order.

```
SELECT customer_id, order_date, amount,
       LAG(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_amount
FROM Orders;
```

Q How would you compute a running total (cumulative sum) of order amounts by date?

A:

```
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM Orders;
```

7 Set Operations

Q Explain UNION, UNION ALL, INTERSECT, and EXCEPT / MINUS.

A: UNION combines two result sets and removes duplicate rows (which requires an implicit sort/dedup, so it has overhead). UNION ALL combines them and keeps duplicates — faster since no dedup step is needed, and preferred whenever you know the sets are already disjoint or you don't care about duplicates. INTERSECT returns only rows present in both result sets. EXCEPT (SQL Server/PostgreSQL) or MINUS (Oracle) returns rows in the first result set that are not in the second.

8 Normalization & Database Design

Q What is normalization, and why do we do it?

A: Normalization is the process of organizing tables to reduce data redundancy and avoid update/insert/delete anomalies, by splitting data into related tables connected via foreign keys instead of repeating it. It trades some query simplicity (you now need joins) for data integrity — you only store a fact like a customer's address in one place, so it can't go out of sync between rows.

Q Explain 1NF, 2NF, and 3NF with a concrete example.

A:

1NF (First Normal Form): Every column holds atomic (indivisible) values, no repeating groups. A column storing "Java, Python, C++" as one comma-separated string violates 1NF — it should be a separate row per skill.

2NF: Must be in 1NF, and every non-key column must depend on the *whole* primary key, not just part of it (relevant only for composite keys). E.g., in `Enrollments(student_id, course_id, student_name)`, `student_name` depends only on `student_id`, not the full composite key — that's a 2NF violation; `student_name` belongs in a separate `Students` table.

3NF: Must be in 2NF, and no non-key column depends on another non-key column (no transitive dependency). E.g., storing both `dept_id` and `dept_name` on the `Employees` table is a 3NF violation, since `dept_name` depends on `dept_id`, not on `emp_id` — it belongs in a separate `Departments` table.

Q What is denormalization, and when would you intentionally do it?

A: Denormalization intentionally reintroduces redundancy — e.g., storing a precomputed total or duplicating a rarely-changing field — to avoid expensive joins on read-heavy, performance-critical paths (like an analytics dashboard or a reporting table). It's a deliberate tradeoff: faster reads, at the cost of extra storage and more careful application logic to keep the duplicated data in sync.

Q How would you design a schema for a many-to-many relationship, e.g., students and courses?

A: Use a junction (bridge/associative) table between them — `Enrollments(student_id, course_id, enrolled_date)` — with a composite primary key (or its own surrogate ID) and foreign keys to both `Students` and `Courses`. Neither `Students` nor `Courses` stores a direct reference to the other; the junction table represents each individual relationship as its own row, which also gives you a natural place to store relationship-specific data like an enrollment date or grade.

9 Indexes & Query Optimization

Q What is an index, and how does it speed up queries?

A: An index is an auxiliary data structure — typically a B-tree — built on one or more columns, that lets the database locate matching rows via an efficient lookup instead of scanning every row (a full table scan). Without an index, finding a row by email means checking every row, $O(n)$; with a B-tree index, it's closer to $O(\log n)$.

Q What's the difference between a clustered and a non-clustered index?

A: A **clustered index** determines the physical order rows are stored on disk — there can be only one per table (usually the primary key). A **non-clustered index** is a separate structure that stores the indexed column(s) plus a pointer back to the actual row — a table can have many of these. Because a non-clustered index requires that extra lookup step to fetch the full row, it's slightly slower than reading directly from a clustered index for the indexed columns, but cheap to add for whichever columns you filter/sort on often.

Q What are the downsides of adding indexes? Should you index every column?

A: No — every index adds storage overhead and slows down writes (INSERT/UPDATE/DELETE), because the index has to be updated alongside the table data. I'd index columns that are frequently used in WHERE clauses, JOIN conditions, or ORDER BY — not columns rarely queried, and not columns with very low cardinality (like a boolean flag) where an index doesn't help the optimizer much anyway.

Q How would you diagnose and fix a slow query?

A: First, run `EXPLAIN` (or `EXPLAIN ANALYZE`) to see the query execution plan — it shows whether the database is doing a full table scan versus using an index, the estimated row counts at each step, and where most of the cost is. Common fixes: add a missing index on a filtered/joined column, rewrite a correlated subquery as a join, avoid `SELECT *` when only a few columns are needed, avoid applying functions to indexed columns in `WHERE` (e.g. `WHERE YEAR(order_date) = 2026` can't use an index on `order_date` — rewrite as a date range instead), and check if statistics are stale so the optimizer's row estimates are inaccurate.

Q What is a covering index?

A: An index that includes every column a query needs (both filter and selected columns), so the database can satisfy the whole query from the index alone without a lookup back to the actual table rows — this is significantly faster for read-heavy queries.

10 Transactions, ACID & Locking

Q Explain the ACID properties.

A:

Atomicity: a transaction's operations either all succeed or all fail together — no partial updates.

Consistency: a transaction moves the database from one valid state to another, never violating constraints.

Isolation: concurrent transactions don't see each other's uncommitted intermediate changes.

Durability: once committed, changes survive even a crash immediately after — they're persisted to disk.

Q What is a transaction, and how do COMMIT / ROLLBACK / SAVEPOINT work?

A: A transaction is a group of one or more SQL statements executed as a single atomic unit, started implicitly or with `BEGIN TRANSACTION`. `COMMIT` makes all changes in the transaction permanent; `ROLLBACK` undoes everything since the transaction began. `SAVEPOINT` marks an intermediate point inside a transaction that you can roll back to without discarding the entire transaction — useful for partial recovery within a larger multi-step operation.

```
BEGIN;
UPDATE Accounts SET balance = balance - 100 WHERE id = 1;
SAVEPOINT after_debit;
UPDATE Accounts SET balance = balance + 100 WHERE id = 2;
-- if something goes wrong here:
ROLLBACK TO after_debit;
COMMIT;
```

Q What are the transaction isolation levels, and what problem does each solve?

A:

Level	Prevents	Still allows
Read Uncommitted	—	Dirty reads, non-repeatable reads, phantom reads
Read Committed	Dirty reads	Non-repeatable reads, phantom reads
Repeatable Read	Dirty + non-repeatable reads	Phantom reads
Serializable	All of the above	Nothing — fully isolated, but slowest

Dirty read: reading another transaction's uncommitted changes. **Non-repeatable read:** re-reading the same row twice in one transaction gives different values because another transaction updated it in between. **Phantom read:** re-running the same range query gives a different set of rows because another transaction inserted/deleted matching rows in between.

Q What is a deadlock, and how would a database (or you) resolve one?

A: A deadlock happens when two transactions each hold a lock the other needs, and each is waiting on the other — neither can proceed. The database's deadlock detector picks one transaction as the "victim," rolling it back and releasing its locks so the other can continue; the victim's application code then needs to retry. To reduce deadlocks in application code, I'd keep transactions short, and always acquire locks on multiple resources in a consistent order across the codebase.

11 Views, Stored Procedures, Triggers & Functions

Q What is a view, and how is it different from a table?

A: A view is a saved, named SELECT query that behaves like a virtual table — it doesn't store data itself (a regular view just re-runs its underlying query each time it's referenced), but it simplifies repeated complex queries and can restrict which columns/rows a user is allowed to see. A **materialized view** (PostgreSQL, Oracle) is the exception — it does physically store the query's result and must be manually or periodically refreshed, trading staleness for speed.

Q What's the difference between a stored procedure and a function?

A: A **function** must return a value and can be used inline within a query (e.g., in SELECT or WHERE); it generally can't modify data or manage transactions. A **stored procedure** may or may not return a value, is called on its own (not embedded in a query expression), can perform DML/DDDL, manage its own transactions, and can have both input and output parameters.

Q What is a trigger? Give a practical example.

A: A trigger is code that automatically runs in response to a specific DML event (INSERT/UPDATE/DELETE) on a table, either BEFORE or AFTER the event. Example: an **AFTER INSERT** trigger on an Orders table that automatically decrements stock in an Inventory table, keeping the two in sync without relying on application code to remember to do it. They're powerful but can make data flow harder to trace/debug, so I use them sparingly and document them clearly.

12 NULL Handling, Strings & Dates

Q Why does `NULL = NULL` evaluate to false (not true) in SQL?

A: `NULL` represents "unknown," not a value — so comparing two unknowns with `=` doesn't yield true, it yields `UNKNOWN` (which acts like false for filtering purposes). That's why you must use `IS NULL` / `IS NOT NULL` instead of `= NULL` to test for nulls.

Q What does `COALESCE()` do, and how is it different from `IFNULL()` / `ISNULL()` ?

A: `COALESCE(a, b, c, ...)` is the ANSI-standard function that returns the first non-`NULL` value from a list of any length — useful for fallback chains, e.g. `COALESCE(nickname, first_name, 'Unknown')`.

`IFNULL()` (MySQL) and `ISNULL()` (SQL Server) are vendor-specific two-argument equivalents. I default to `COALESCE` for portability.

Q Common string and date functions you should know for interviews.

A:

Task	Example
Concatenate strings	<code>CONCAT(first_name, ' ', last_name)</code>
Substring	<code>SUBSTRING(name, 1, 3)</code>
Trim whitespace	<code>TRIM(name)</code>
Change case	<code>UPPER(name)</code> / <code>LOWER(name)</code>
Pattern match	<code>name LIKE 'A%'</code> (starts with A)
Current date/time	<code>CURRENT_DATE</code> , <code>NOW()</code>
Extract date part	<code>EXTRACT(YEAR FROM order_date)</code>
Date difference	<code>DATEDIFF(day, start_date, end_date)</code> (SQL Server)

13 SQL Injection & Security

Q What is SQL injection, and how do you prevent it?

A: SQL injection happens when untrusted user input is concatenated directly into a SQL query string, letting an attacker inject their own SQL logic — e.g., a login form building `"SELECT * FROM users WHERE email = '" + input + "'"`, where input `' OR '1'='1` bypasses the check entirely. The fix is **parameterized queries / prepared statements**, where input is passed as a bound parameter rather than being interpolated into the SQL string — the database driver handles escaping so user input can never be interpreted as SQL syntax. I'd also apply least-privilege database accounts and validate/sanitize input as defense in depth, but parameterized queries are the actual fix.

```
-- Vulnerable (string concatenation):
"SELECT * FROM users WHERE email = '" + userInput + "'"

-- Safe (parameterized):
SELECT * FROM users WHERE email = ?    -- or $1 / :email depending on driver
```

14 SQL vs NoSQL

Q When would you choose a relational (SQL) database over a NoSQL one, and vice versa?

A: Relational databases fit data with clear structure and relationships where consistency and complex queries (joins, transactions across multiple tables) matter — financial records, order systems, anything needing strong ACID guarantees. NoSQL (document/key-value/wide-column) fits data that's naturally flexible or nested, needs to scale horizontally across many servers, or has very high write throughput where eventual consistency is acceptable — e.g., a chat app's messages, session/cache data, or catalogs with widely varying attributes per item. In practice, plenty of systems use both: PostgreSQL for the transactional core, MongoDB or Redis alongside it for specific access patterns.

15 Classic Query-Writing Problems (With Solutions)

Q Find the Nth highest salary overall.

A: Using window functions (works with ties correctly):

```
SELECT salary FROM (  
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk  
  FROM Employees  
) t WHERE rnk = 3; -- 3rd highest, change N as needed
```

Classic alternative without window functions (works in older MySQL):

```
SELECT DISTINCT salary FROM Employees e1  
WHERE 2 = (  
  SELECT COUNT(DISTINCT salary) FROM Employees e2 WHERE e2.salary >= e1.salary  
);
```

Q Find duplicate rows in a table (e.g., duplicate emails).

A:

```
SELECT email, COUNT(*) AS occurrences  
FROM Customers  
GROUP BY email  
HAVING COUNT(*) > 1;
```

Q Delete duplicate rows, keeping only one copy of each.

A:

```
DELETE FROM Customers  
WHERE customer_id NOT IN (  
  SELECT MIN(customer_id)  
  FROM Customers  
  GROUP BY email  
);
```

Q Find employees who earn more than their manager.

A:

```
SELECT e.name AS employee, e.salary, m.name AS manager, m.salary AS manager_salary
FROM Employees e
JOIN Employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Q Find the top 3 highest-paid employees in each department.

A:

```
SELECT emp_id, name, dept_id, salary FROM (
  SELECT emp_id, name, dept_id, salary,
         DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
  FROM Employees
) t
WHERE rnk <= 3;
```

Q Find customers who have never placed an order.

A:

```
SELECT c.customer_id, c.name
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

-- Equivalent using NOT EXISTS (often the better performer):
SELECT c.customer_id, c.name
FROM Customers c
WHERE NOT EXISTS (
  SELECT 1 FROM Orders o WHERE o.customer_id = c.customer_id
);
```

Q Find the month with the highest total order revenue.

A:

```
SELECT EXTRACT(YEAR FROM order_date) AS yr,
       EXTRACT(MONTH FROM order_date) AS mo,
       SUM(amount) AS total_revenue
FROM Orders
GROUP BY yr, mo
ORDER BY total_revenue DESC
LIMIT 1;
```

Q Calculate month-over-month percentage growth in revenue.

A:

```
WITH monthly AS (  
  SELECT DATE_TRUNC('month', order_date) AS mo, SUM(amount) AS revenue  
  FROM Orders  
  GROUP BY mo  
)  
SELECT mo, revenue,  
  LAG(revenue) OVER (ORDER BY mo) AS prev_revenue,  
  ROUND(  
    (revenue - LAG(revenue) OVER (ORDER BY mo)) * 100.0  
    / LAG(revenue) OVER (ORDER BY mo), 2  
  ) AS pct_growth  
FROM monthly;
```

Q Pivot rows into columns — e.g., total sales per city as separate columns.

A: Standard SQL uses conditional aggregation (works everywhere); some databases (SQL Server, Oracle) also offer a native `PIVOT` keyword.

```
SELECT  
  SUM(CASE WHEN c.city = 'Delhi' THEN o.amount ELSE 0 END) AS delhi_sales,  
  SUM(CASE WHEN c.city = 'Mumbai' THEN o.amount ELSE 0 END) AS mumbai_sales  
FROM Orders o  
JOIN Customers c ON o.customer_id = c.customer_id;
```

Q Find consecutive days a user was active (a "streak" problem).

A: The classic trick: subtract a row number from the date — rows in the same streak will produce the same resulting "group" value.

```
WITH numbered AS (  
  SELECT user_id, activity_date,  
    activity_date - (ROW_NUMBER() OVER (  
      PARTITION BY user_id ORDER BY activity_date  
    ) || ' days')::interval AS grp  
  FROM UserActivity  
)  
SELECT user_id, MIN(activity_date) AS streak_start, MAX(activity_date) AS streak_end,  
  COUNT(*) AS streak_length  
FROM numbered  
GROUP BY user_id, grp  
ORDER BY streak_length DESC;
```

Q Swap values in two columns for all rows without using a temp variable (classic puzzle, e.g. swap 'M'/'F' in a gender column).

A:

```
UPDATE Users  
SET gender = CASE gender  
  WHEN 'M' THEN 'F'  
  WHEN 'F' THEN 'M'  
END;
```

16 Glossary — Key SQL Terms

Term	Plain-Language Explanation
Schema	The structural blueprint of a database — its tables, columns, types, and relationships.
Primary Key	The column(s) that uniquely identify each row in a table; cannot be NULL.
Foreign Key	A column referencing another table's primary key, enforcing that the relationship is valid.
Normalization	Organizing tables to eliminate redundant data and prevent update anomalies.
Index	An auxiliary structure (usually a B-tree) that speeds up row lookup on specific columns.
B-tree	A balanced, sorted tree data structure most database indexes are built on for fast lookups.
Transaction	A group of statements executed as one atomic, all-or-nothing unit of work.
ACID	Atomicity, Consistency, Isolation, Durability — the guarantees a reliable transaction provides.
Isolation Level	How much one transaction is shielded from the effects of concurrent transactions.
Deadlock	Two transactions each waiting on a lock the other holds, unable to proceed.
View	A saved, named query that behaves like a virtual (usually non-materialized) table.
Materialized View	A view whose result is physically stored and periodically refreshed, for faster reads.
Stored Procedure	Precompiled, reusable SQL/procedural code stored and executed inside the database.
Trigger	Code that runs automatically in response to an INSERT/UPDATE/DELETE event on a table.
CTE	Common Table Expression — a named, temporary result set defined with WITH, scoped to one query.
Recursive CTE	A CTE that references itself to walk hierarchical or graph-like data (e.g., an org chart).
Window Function	A function computing a value across a set of related rows without collapsing them (unlike GROUP BY).
Correlated Subquery	A subquery that references a column from its outer query, conceptually re-run per outer row.
Cartesian Product	Every row of one table paired with every row of another (result of a CROSS JOIN).
Referential Integrity	The guarantee that foreign key values always correspond to an existing row in the referenced table.
SQL Injection	An attack that injects malicious SQL via unsanitized user input concatenated into a query string.
Parameterized Query	A query with placeholder parameters bound separately from the SQL text, preventing injection.
EXPLAIN Plan	The database's report of how it intends to execute a query — used to diagnose slow queries.
Sharding	Splitting a database's data horizontally across multiple servers to scale writes/storage.

Replication

Copying data to additional database servers for redundancy and to scale read traffic.

Final tip: If asked to write a query live, say your approach out loud before typing — "I'll join Orders to Customers, filter by date, then aggregate by customer" — interviewers weight your reasoning as much as syntax correctness, and it lets them redirect you before you've written the wrong query end to end.